

# FinRAG: Eval-Gated Hybrid RAG over SEC Filings

*Implementation plan for AI-assisted development. Author: Ali.*

## 1. Project summary

A hybrid agentic RAG system that answers questions about SEC 10-K filings. Every change to prompts, chunking, embeddings or retrieval is tested against a verified golden set, and a GitHub Actions check blocks the merge if quality regresses.

**The evaluation harness is the product.** The retrieval stack is the thing under test.

**Success criteria** (all must be true before calling the project done):

- A reproducible corpus: about 6 companies, 3 fiscal years each, downloaded by script with a manifest.
- A golden set of about 100 verified questions, with a held-out split that was run once.
- An ablation table: dense only, BM25 only, hybrid, hybrid plus rerank, plus routing.
- A CI gate that fails on a deliberately bad change (demo PR kept in the repo history).
- A README that leads with measured results, a failure analysis, and a live demo link.

Every number reported must come from a real run logged in `EXPERIMENTS.md`. Never estimate or invent metrics.

## 2. How to work with the AI assistant

These rules go into `CLAUDE.md` at the repo root (template in section 5) so every session follows them.

1. **One step at a time.** Work only on the current step in `PROGRESS.md`. Do not start the next step until the current step's "Done when" conditions are met and logged.
2. **Plan before code.** At the start of a step, the AI restates the goal, lists the files it will create or change, and waits for approval if the change touches architecture.
3. **Tests with code.** Each module ships with tests. A step is not done while tests fail.
4. **Log as you go.** Update `PROGRESS.md` at the end of every session, add every problem to `PROBLEMS.md` the moment it is found (not later), and record every experiment in `EXPERIMENTS.md`.

5. **Ask before big decisions.** Changing the stack, schema, or eval design needs an entry in `DECISIONS.md` first.
6. **Never touch the held-out split** until Step 13. Never tune on it, never print its scores before then.
7. **Never fabricate.** If a number, API behavior or library feature is uncertain, check the current documentation or run it and say what was verified.
8. **Small commits.** One logical change per commit, message references the step (for example `step-05: add table-aware chunker`).

### 3. Stack and key decisions

| Concern             | Choice                                                                                       | Reason                                                                  |
|---------------------|----------------------------------------------------------------------------------------------|-------------------------------------------------------------------------|
| Language            | Python 3.11+                                                                                 | Ecosystem for retrieval and eval                                        |
| Package manager     | <code>uv</code> or <code>pip</code> + <code>requirements.txt</code>                          | Reproducible installs                                                   |
| Storage and vectors | Postgres with <code>pgvector</code> (Docker locally, Supabase or similar hosted)             | You already know Postgres and Supabase                                  |
| Sparse search       | <code>bm25s</code> or <code>rank_bm25</code> first; consider Postgres full-text search later | Simple to start; note Postgres FTS ranking is not identical to BM25     |
| Embeddings          | A BGE model run locally, or a hosted embedding API                                           | Local is free and reproducible; pick one and pin it                     |
| Reranker            | <code>BAAI/bge-reranker-base</code> locally, or Cohere Rerank                                | Local keeps CI cost at zero                                             |
| Generation          | One hosted LLM, temperature 0, pinned model version                                          | Reproducibility                                                         |
| Orchestration       | Plain Python state machine first; LangGraph optional                                         | Easier to explain and debug; add a framework only if it earns its place |
| Eval                | Own harness for retrieval and numeric checks; Ragas or DeepEval for LLM-judge metrics        | Deterministic checks gate first                                         |

| Concern    | Choice                                                   | Reason                |
|------------|----------------------------------------------------------|-----------------------|
| API and UI | FastAPI plus a minimal UI (Streamlit or small React app) | Fast demo             |
| CI         | GitHub Actions                                           | Required for the gate |

Library APIs change often. Before using Ragas, DeepEval, LangGraph or any SDK, the AI must read the current docs for the installed version instead of relying on memory.

## 4. Repository layout

```

finrag/
  CLAUDE.md          # rules for the AI assistant
  PROGRESS.md        # living status tracker
  PROBLEMS.md        # problem and fix log
  DECISIONS.md       # short decision records
  EXPERIMENTS.md     # every metric run, with config
  README.md
  pyproject.toml
  docker-compose.yml # postgres + pgvector
  data/
    manifest.json    # companies, filings, URLs, checksums
    raw/             # downloaded filings (gitignored)
    parsed/          # parsed elements (gitignored)
    xbrl/            # extracted facts (small, may be committed)
  golden/
    golden_v1.jsonl  # all questions
    README.md        # schema and authoring rules
  src/finrag/
    ingest/          # edgar download, html parsing
    xbrl/            # fact extraction
    chunking/        # table-aware chunker
    index/           # pgvector and BM25 indexing
    retrieval/       # dense, sparse, RRF, reranker
    agent/           # router, rewriter, decomposer
    generate/        # prompts, structured output, refusal
    api/            # FastAPI app
  eval/
    metrics.py       # recall@k, MRR, numeric match
    judge.py         # LLM-judge wrapper
    run_eval.py      # CLI: run a config on a split

```

```

baseline.json      # committed baseline scores
judge_validation/  # hand labels and agreement script
tests/
.github/workflows/
eval.yml
smoke.yml

```

## 5. Files the AI must maintain

### 5.1 CLAUDE.md

```

# FinRAG: assistant guide

## Project
Hybrid RAG over SEC 10-K filings with an eval-gated CI pipeline.
The evaluation harness is the product.

## Rules
1. Work only on the current step in PROGRESS.md. One step at a time.
2. At the start of a step: restate the goal, list files to touch,
   wait for approval if architecture changes.
3. Every module ships with tests. A step is done only when its
   "Done when" checklist passes.
4. Update PROGRESS.md at the end of every session.
5. Log every problem in PROBLEMS.md immediately, with cause and fix.
6. Log every metric run in EXPERIMENTS.md with config and commit hash.
7. Record non-trivial choices in DECISIONS.md before implementing.
8. Never read, tune on, or report the held-out split before Step 13.
9. Never invent numbers. Verify library behavior in current docs.
10. Temperature 0, pinned model versions, fixed random seeds.
11. No secrets in the repo. Use environment variables.
12. Respect SEC rules: descriptive User-Agent with contact email,
    stay well under 10 requests per second.

## Conventions
- Type hints and docstrings on public functions.
- Formatting: ruff + black. Tests: pytest.
- Config in YAML files, not hardcoded.
- Commit messages start with the step id, for example step-05.

```

## 5.2 PROGRESS.md template

```
# Progress

**Current step:** 0
**Last updated:** YYYY-MM-DD

## Step status
| Step | Title | Status | Done date |
|---|---|---|---|
| 0 | Bootstrap | not started | |
| 1 | Corpus and downloader | not started | |
| ... | ... | ... | |

Status values: not started, in progress, blocked, done.

## Current session
- Done:
- In progress:
- Next:
- Open questions for Ali:

## Handoff notes
Anything the next session must know (env quirks, half-finished files).
```

## 5.3 PROBLEMS.md template

One entry per problem, newest first.

```
## P-012: Revenue table split across two chunks (Step 5)
- **Date:** YYYY-MM-DD
- **Symptom:** Questions about segment revenue returned the header row only.
- **Root cause:** Chunker split on token count and ignored table boundaries.
- **Fix:** Table-aware chunker keeps tables intact; long tables split by row groups with the header repeated.
- **Verification:** Table-lookup category recall@5 rose from 0.41 to 0.78 (see EXPERIMENTS E-009).
- **Prevention:** Added chunker test with a 60-row table.
- **Status:** resolved
```

The example numbers above are illustrative only. Real entries must cite real runs.

## 5.4 DECISIONS.md template

```
## D-004: Own state machine instead of LangGraph (Step 9)
- **Context:** Router needs three paths.
- **Options:** LangGraph, plain Python state machine.
- **Decision:**
- **Consequences:**
```

## 5.5 EXPERIMENTS.md template

```
## E-009: hybrid + rerank, dev split
- **Date / commit:**
- **Config file:** configs/hybrid_rerank.yaml
- **Split:** dev (n=70)
- **Results:** recall@5, MRR, numeric accuracy, faithfulness, latency
p50/p95, cost/query
- **Per-category notes:**
- **Conclusion and next action:**
```

# 6. Step-by-step plan

Each step lists the goal, tasks, deliverables and a "Done when" checklist. Steps are sequential. The AI logs to `PROGRESS.md`, `PROBLEMS.md` and `EXPERIMENTS.md` throughout.

## Step 0: Bootstrap

**Goal:** a clean repo the AI and CI can both work in.

### Tasks:

- Create the repo, layout from section 4, `pyproject.toml`, `ruff`, `black`, `pytest`, `pre-commit`.
- Add `docker-compose.yml` for Postgres with `pgvector`.
- Write `CLAUDE.md`, `PROGRESS.md`, `PROBLEMS.md`, `DECISIONS.md`, `EXPERIMENTS.md` from the templates.
- Add a trivial passing test and a CI workflow that runs lint and tests.

**Done when:** `docker compose up` starts the database, `pytest` passes locally and in CI, and all five tracking files exist.

## Step 1: Corpus selection and downloader

**Goal:** a reproducible, legal corpus.

**Tasks:**

- Choose one sector and three direct-competitor pairs (for example in semiconductors or cloud software). Prefer companies with different fiscal year ends so you must handle that.
- Write an EDGAR downloader for 10-K filings, 3 fiscal years per company. Send a descriptive `User-Agent` including your contact email and rate-limit well below SEC's published limit of about 10 requests per second.
- Write `data/manifest.json`: company, CIK, fiscal year end, filing date, accession number, URL, SHA-256 checksum.
- Gitignore raw filings. Commit only the manifest.

**Deliverables:** downloader CLI, manifest, tests using a small fixture instead of the live site.

**Done when:** a fresh clone can rebuild `data/raw/` from the manifest and checksums match.

## Step 2: Parse filings into structured elements

**Goal:** turn each filing into typed elements (heading, paragraph, table) with section context.

**Tasks:**

- 10-Ks are HTML (usually inline XBRL), so parse the HTML directly with `lxml` or `BeautifulSoup`. Do not convert to PDF first.
- Detect the standard Items (Item 1A Risk Factors, Item 7 MD&A, Item 8 Financial Statements) and attach the item and heading path to every element.
- Convert each table to a clean structure (list of rows) and a markdown rendering. Capture the units statement (for example "in millions, except per-share data") that sits near the table.
- Store elements as JSONL in `data/parsed/`.

**Done when:** spot-checks of 10 random tables per filing match the original, units are captured for at least 95 percent of financial tables (measure it), and parsing tests pass.

### Step 3: XBRL ground-truth extraction

**Goal:** a trustworthy set of numeric facts for question generation and answer checking.

**Tasks:**

- Pull structured facts from SEC's XBRL company-facts data for each company.
- Filter to 10-K facts, and handle the known pitfalls: the same fact repeats across filings, tag names differ between companies (for example revenue is reported under different tags), values are in raw units while tables show thousands or millions, and fiscal years are labeled differently from calendar years.
- Build a normalized table: company, fiscal year, metric (revenue, net income, operating income, R&D expense, total assets, and so on), value in a single base unit, source tag, filing accession.
- Cross-check at least 20 values by hand against the filing text.

**Done when:** a metrics table covers all company-years for at least 8 metrics, and the hand cross-check passes.

### Step 4: Golden set v1

**Goal:** a verified question set, written before any retrieval tuning.

**Tasks:**

- Define the schema (section 7) and write `golden/README.md` with authoring rules.
- Template-generate numeric questions from the Step 3 table (single-fact, calculation, year-over-year change, cross-company comparison).
- Hand-write about 40 questions across the categories in section 7, recording supporting passages (document, item, page or element id).
- If an LLM drafts questions, review every one and discard weak ones.
- Assign splits: about 70 percent `dev`, 30 percent `heldout`, stratified by category. Lock the held-out file and do not open it again until Step 13.

**Done when:** about 100 questions exist, every one has a reference answer and gold passages (or an explicit refusal expectation), each category has at least 8 questions, and a schema-validation test passes.

### Step 5: Table-aware chunking and indexing

**Goal:** chunks that do not destroy tables, with metadata for filtering.

**Tasks:**

- Chunk by element: paragraphs grouped under their heading; tables kept whole when they fit, otherwise split by row groups with the header repeated.
- Attach metadata to each chunk: company, fiscal year, item, heading path, element id, units, element type.
- Prepend a short context line (company, year, section) to the chunk text used for embedding.
- Index in pgvector (dense) and build the BM25 index. Keep chunking parameters in a config file so CI can detect changes.

**Done when:** every gold passage id maps to at least one chunk, index build is a single command, and chunker tests (including a long-table test) pass.

**Step 6: Baseline pipeline and the eval harness**

**Goal:** a dense-only baseline plus the deterministic half of the harness.

**Tasks:**

- Implement dense retrieval, a generation prompt that requires citations to chunk ids, and structured output (Pydantic) with fields for answer, cited chunks and a refusal flag.
- Implement `eval/metrics.py` : recall@k and MRR against gold passages, and numeric match with tolerance and unit normalization.
- Implement `eval/run_eval.py` : takes a config and a split, writes a per-question results file and a summary.
- Run the baseline on `dev`. Record in `EXPERIMENTS.md` and write the first `baseline.json`.

**Done when:** one command evaluates a config on `dev` and produces per-question and per-category results, and metric functions have unit tests with known inputs.

**Step 7: LLM judge and judge validation**

**Goal:** add faithfulness and answer-correctness metrics, and prove the judge can be trusted.

**Tasks:**

- Integrate Ragas or DeepEval (check current docs for the installed version) or a small custom judge prompt. Use a different model for judging than for generating if possible.

- Hand-label about 30 answers as correct, incorrect or unsupported. Compute agreement with the judge and store it in `eval/judge_validation/`.
- Run the judge several times on the same outputs to measure variance. Store the variance, since the gate tolerance comes from it.

**Done when:** judge agreement and run-to-run variance are measured and logged, and a cost-per-eval-run figure is recorded.

## Step 8: Hybrid retrieval, RRF and reranking

**Goal:** the retrieval tier, with each piece measured.

### Tasks:

- Add BM25 retrieval. Fuse dense and sparse rankings with Reciprocal Rank Fusion (score is the sum of  $1 / (k + \text{rank})$  across lists, with k around 60, tuned on `dev` only if justified).
- Add a cross-encoder reranker over the top fused candidates.
- Run the ablation on `dev` : dense only, BM25 only, hybrid, hybrid plus rerank. Record every run.
- Report per-category results and latency (p50 and p95).

**Done when:** the ablation table exists with real numbers, and `PROBLEMS.md` holds at least the real issues you hit (do not pad it).

## Step 9: Routing and query transformation

**Goal:** test whether an agentic layer actually helps.

### Tasks:

- Build a small state machine with three paths: answer directly, rewrite an ambiguous query, or decompose a multi-hop query into sub-queries whose results are combined.
- Write routing tests with hand-labeled example queries.
- Add a simpler comparison arm: a single retrieval pass with no router.
- Run the ablation again. **Keep the router only if the dev numbers justify it.** If it does not help, log the result honestly and either simplify or remove it.

**Done when:** the router-versus-no-router comparison is logged, and `DECISIONS.md` records what was kept and why.

## Step 10: Reliability and refusal behavior

**Goal:** behave safely when the corpus cannot answer.

**Tasks:**

- Enforce structured output with validation and retry on malformed responses.
- Add exponential backoff, a timeout, and a fallback model.
- Add refusal behavior for unanswerable questions, and premise correction for false-premise questions (for example a question asserting revenue fell when it rose).
- Add refusal and premise-correction metrics to the harness.

**Done when:** unanswerable and false-premise categories have measured scores and a failure-injection test (simulated API errors) passes.

## Step 11: CI gate

**Goal:** the showstopper feature.

**Tasks:**

- Create `eval.yml`, triggered on pull requests that touch `src/finrag/chunking`, `retrieval`, `agent`, `generate`, prompts, or config (use `paths: filters`), plus a scheduled nightly run.
- Gate in tiers: (1) retrieval metrics, deterministic, run first; (2) numeric match; (3) judge metrics, only if tiers 1 and 2 pass.
- Pass rule: fail if any metric is below the committed `baseline.json` minus a tolerance derived from the variance measured in Step 7. Do not use a fixed number such as 85 percent unless it comes from your measurements.
- Post a per-question markdown report as a PR comment and upload results as an artifact.
- Add `smoke.yml`: about 15 questions on every push.
- Store API keys in GitHub Secrets. Cache embeddings and unchanged results to control cost.
- Document the process for intentionally updating the baseline in the same PR.

**Done when:** a normal PR passes, and the gate's behavior is documented.

## Step 12: Demo PR, API and deployment

**Goal:** prove the gate works and give recruiters something to click.

**Tasks:**

- Open a PR with a deliberately bad change (for example chunking that splits tables). Confirm the gate blocks it. Keep the PR open or linked, and save the failing report as a screenshot.
- Build the FastAPI endpoint and a minimal UI that shows the answer, cited passages and a refusal when appropriate.
- Deploy it, add a CI status badge to the README, and keep API usage bounded (rate limits and a spending cap).

**Done when:** the blocked PR is linked from the README and the demo is reachable.

## Step 13: Held-out run, report and README

**Goal:** final, honest results.

**Tasks:**

- Freeze the configuration. Run the held-out split **once** on each ablation arm. Do not tune afterward.
- Write the final results into the README: ablation table, per-category breakdown, judge agreement, cost and latency, and a failure analysis drawn from `PROBLEMS.md`.
- Write a short blog post: what failed in naive RAG on this data, and what fixed it.
- Prepare interview notes: explain RRF, why a cross-encoder beats a bi-encoder, chunking choices, and how you set the gate tolerance.

**Done when:** every number in the README traces to a logged run, and a stranger can reproduce the pipeline from the README.

## 7. Golden set schema and categories

One JSON object per line in `golden/golden_v1.jsonl`:

```
{
  "id": "q_017",
  "category": "multi_hop_cross_company",
  "question": "Which of Company A and Company B had the higher operating margin in FY2024?",
  "reference_answer": "Company A (X percent vs Y percent).",
  "answer_type": "numeric | short_text | free_text | refusal | premise_correction",
  "numeric_answer": { "value": 0.0, "unit": "percent", "tolerance": 0.1 },
  "gold_passages": ["companyA_fy2024_item7_t12", "companyB_fy2024_item7_t09"],
  "source": "xbrl_template | handwritten | llm_drafted_reviewed",
  "split": "dev | heldout",
  "notes": ""
}
```

| Category                | What it tests                                           |
|-------------------------|---------------------------------------------------------|
| single_fact             | Basic retrieval                                         |
| table_lookup            | Chunking and table handling                             |
| narrative               | Prose retrieval (for example reasons for margin change) |
| multi_hop_cross_year    | Decomposition                                           |
| multi_hop_cross_company | Decomposition and synthesis                             |
| calculation             | Retrieval plus arithmetic                               |
| ambiguous               | Query rewriting or clarification                        |
| restated_or_conflicting | Synthesis and citation                                  |
| false_premise           | Resisting the user’s framing                            |
| unanswerable            | Knowing when to refuse                                  |

Use at least 8 questions per category so per-category results mean something.

## 8. Evaluation and gating design

### Metrics by tier

1. **Retrieval (deterministic, free):** recall@k and MRR against `gold_passages`.
2. **Answer correctness (deterministic where possible):** numeric match with tolerance and unit normalization; exact or fuzzy match on short text.
3. **Judge (noisy, costs money):** faithfulness and answer correctness on free-text answers, plus refusal and premise-correction checks.

**Operational metrics:** latency p50 and p95, tokens, and cost per query.

### Gate rule

```
fail if retrieval_recall < baseline.retrieval_recall - tol_retrieval
fail if numeric_accuracy < baseline.numeric_accuracy - tol_numeric
fail if judge_faithfulness < baseline.judge_faithfulness - tol_judge
```

Tolerances come from measured run-to-run variance. Retrieval and numeric checks run first so most regressions fail cheaply.

### Integrity rules

- Develop only on `dev`. Report `heldout` once.
- Pin model versions, use temperature 0, and fix seeds.
- Include a **no-retrieval baseline** (the model answers with no documents) to show retrieval is doing the work, because models may remember well-known companies' numbers.
- Report judge agreement with your hand labels.

## 9. Known risks and how to handle them

| Risk                                        | Mitigation                                                                    |
|---------------------------------------------|-------------------------------------------------------------------------------|
| Units and scale (thousands versus millions) | Capture the units statement in parsing and chunk metadata; test it explicitly |
| Fiscal versus calendar years                | Store fiscal year end per company; normalize in the metrics table             |
| XBRL tag differences between companies      | Map tags per company and cross-check by hand                                  |
| Tables split across chunks                  | Table-aware chunker with a dedicated test                                     |

| Risk                                   | Mitigation                                                                        |
|----------------------------------------|-----------------------------------------------------------------------------------|
| Judge noise and CI flakiness           | Measure variance; derive tolerances; deterministic tiers first                    |
| CI cost                                | Path filters, smoke set, caching                                                  |
| Memorization of famous companies       | No-retrieval baseline; emphasize recent fiscal years and table-specific questions |
| Overfitting to dev                     | Stratified held-out split run once                                                |
| Router adds complexity without benefit | Ablation decides; remove it if it does not help                                   |
| Scope creep                            | Extensions (guardrails repo, second corpus, MCP wrapper) only after Step 13       |

## 10. Final README checklist

- ☐ One-sentence pitch and a live demo link
- ☐ CI badge and a link to the blocked demo PR
- ☐ Ablation table with real numbers (recall@k, numeric accuracy, faithfulness, latency, cost per query)
- ☐ Per-category breakdown
- ☐ Judge validation line (agreement on about 30 hand-labeled samples)
- ☐ Failure analysis of 3 to 5 real problems from `PROBLEMS.md`
- ☐ Reproduction instructions (manifest, one-command index build, one-command eval)
- ☐ Honest limitations section
- ☐ Resume bullet filled with measured numbers only